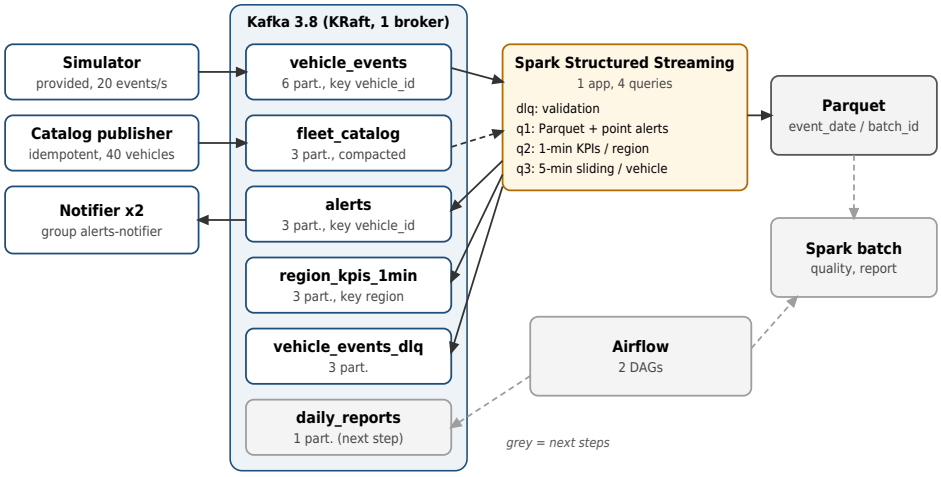
Real-Time Fleet Monitoring Platform

Progress review · Big Data Frameworks I (ST2DEI-DE)

Nathan CHEVAUCHE · Benjamin TONNEAU · promo 2028

1. Architecture



The whole platform runs in Docker and starts with `docker compose up -d`: Kafka broker, init container (topics + catalog), simulator, two notifiers and the Spark streaming job. The Parquet files are the hand-off to the batch side (next steps).

2. Kafka layer

Topic	Part.	Key	Role
vehicle_events	6	vehicle_id	raw telemetry, order kept per vehicle
fleet_catalog	3	vehicle_id	reference data, compacted: latest record per vehicle
alerts	3	vehicle_id	alerts from Spark, read by the notifiers
region_kpis_1min	3	region	1-minute regional indicators
vehicle_events_dlq	3	original key	invalid messages with a reason

- **Catalog publisher**: 40 vehicles published once with an idempotent producer.
- **Alerts notifier**: consumer group `alerts-notifier`, two instances, offsets committed manually after each alert is handled, `cooperative-sticky` assignment (when an instance stops, its partitions move to the other one).
- **Semantics**: idempotent producers, at-least-once on the alert path (commit after processing).

3. Spark Structured Streaming

One application, four streaming queries, each with its own checkpoint:

- **Validation**: explicit schema; broken JSON, missing field, bad timestamp or speed outside 0-250 km/h -> `vehicle_events_dlq` with the reason.
- **Deduplication** on `event_id` with a 2-minute watermark, then **enrichment** with the catalog (broadcast join); a vehicle missing from the catalog gets the region UNKNOWN.
- **Query 1**: events written as Parquet partitioned by `event_date` and `batch_id`, point alerts (speed above 120 km/h or harsh braking) published to `alerts`.
- **Query 2**: 1-minute tumbling windows per region -> `region_kpis_1min`.
- **Query 3**: 5-minute windows sliding every minute per vehicle -> `sustained_speeding` alerts.

4. Status

- **Done**: Docker infrastructure, Kafka topics and catalog, notifier, streaming job with the four queries, running end to end.
- **Next**: Spark batch jobs (quality, daily report, skew), Airflow DAGs, failure experiments and performance study, report.